

L43 — Collision Resolution: Chaining and Linear Probing

Unit: 3 | Chapter: Hashing | BT Level: BT5 | CO: CO2, CO3

Learning Objectives

- Explain the inevitability of collisions using the **Pigeonhole Principle** and **Birthday Paradox**
 - Distinguish between **Open Hashing** (Separate Chaining) and **Closed Hashing** (Open Addressing)
 - Analyze **Separate Chaining** time complexities under the **Simple Uniform Hashing Assumption (SUHA)**
 - Apply and contrast open addressing probing techniques: **Linear Probing**, **Quadratic Probing**, and **Double Hashing**
 - Analyze **Primary Clustering** and **Secondary Clustering** phenomena
 - Explain the role of **Tombstones (Lazy Deletion)** in preserving open addressing search invariants
 - Trace insertion sequences and compute probe counts for typical GATE numerical problems
-

1. The Collision Inevitability Principle

A collision occurs when two distinct keys map to the same table slot: $h(k_1) = h(k_2)$ for $k_1 \neq k_2$.

1.1 The Pigeonhole Principle

Let U be the universe of possible keys and m be the number of hash table slots. Because $|U| > m$, Dirichlet's **Pigeonhole Principle** guarantees that no matter what hash function $h: U \rightarrow \{0, 1, \dots, m-1\}$ is chosen, collisions are mathematically unavoidable.

1.2 The Birthday Paradox Intuition

Even when the number of inserted keys n is substantially smaller than m , collisions occur much earlier than intuition suggests.

Under uniform hashing, the probability that n randomly chosen keys do not collide is:

$$P(\text{no collision}) = \prod_{k=1}^{n-1} \left(1 - \frac{k}{m}\right) \approx e^{-n^2 / (2m)}$$

Setting $P(\text{collision}) \geq 0.5$ gives:

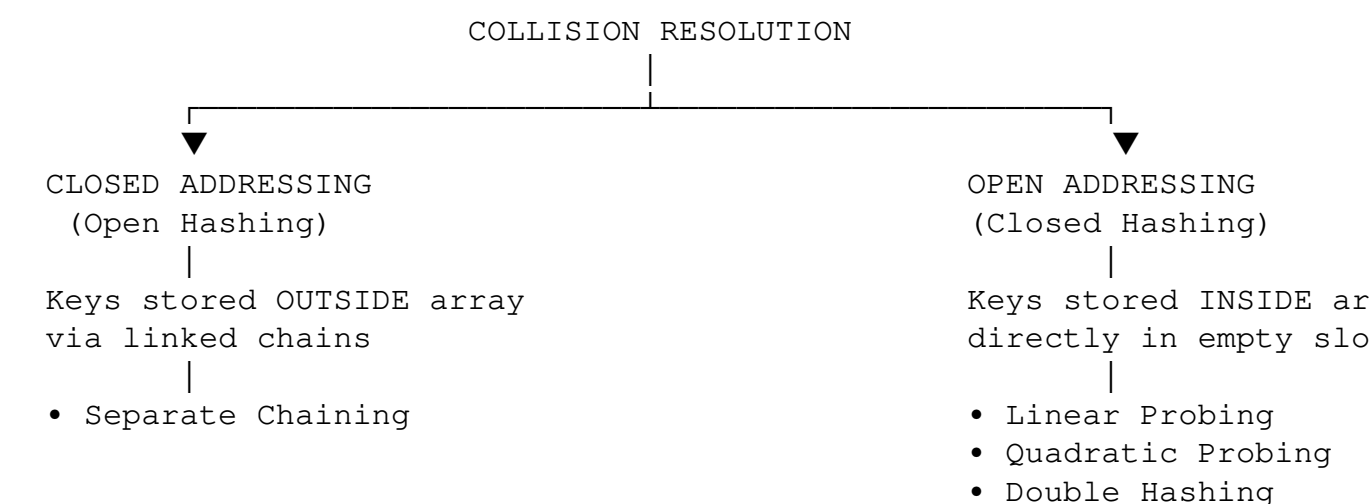
$$n \approx \sqrt{2m \ln 2} \approx 1.177 \sqrt{m}$$

Table Size (\$m\$)	Insertions (\$n\$) for $P(\text{collision}) \geq 50\%$	Implication
$m = 365$ (Days in year)	$n = 23$	Classic Birthday Problem
$m = 1,000$ slots	$n = 38$ keys	Table barely 3.8% full
$m = 10,000$ slots	$n = 118$ keys	Collisions begin almost immediately

[!IMPORTANT] Because collisions are guaranteed in any practical hash table, the efficiency of a table depends entirely on its **collision resolution strategy**.

2. Terminology Demystification (GATE Focus)

Computer science literature uses opposing terms that frequently appear in GATE questions:



Strategy	Synonyms	Where Keys Reside	Load Factor Constraint
Closed Addressing	Open Hashing , Separate Chaining, Direct Chaining	Outside the array in auxiliary linked lists	Unbounded ($\alpha > 1$ permitted)
Open Addressing	Closed Hashing , Probing Hashing	Inside the array itself in vacant slots	Strictly $\alpha \leq 1$

- **Why "Open Hashing"?** Auxiliary memory is *open* to grow dynamically beyond table size m .
- **Why "Closed Addressing"?** A key's home address $h(k)$ is *closed* (fixed); it only lives in chain $h(k)$.
- **Why "Open Addressing"?** The address where a key ends up is *open* to change via probing.
- **Why "Closed Hashing"?** Storage is strictly *closed* within the bounds of array indices $0 \dots m-1$.

3. Strategy 1: Separate Chaining (Open Hashing)

In **Separate Chaining**, each table slot acts as a bucket header pointing to a linked list of

records that hash to that slot.

Table Size $m = 7$, Keys: [10, 22, 31, 4, 15, 28, 17], $h(k) = k \% 7$

Slot 0: [•] $\longrightarrow$ [28] $\longrightarrow$ None
Slot 1: [•] $\longrightarrow$ [15] $\longrightarrow$ [22] $\longrightarrow$ None
Slot 2: [•] $\longrightarrow$ None (Empty)
Slot 3: [•] $\longrightarrow$ [17] $\longrightarrow$ [31] $\longrightarrow$ [10] $\longrightarrow$ None
Slot 4: [•] $\longrightarrow$ [4] $\longrightarrow$ None
Slot 5: [•] $\longrightarrow$ None (Empty)
Slot 6: [•] $\longrightarrow$ None (Empty)

3.1 Load Factor (α)

The load factor is defined as:

$$\alpha = \frac{n}{m}$$

where n is the total number of stored keys and m is the number of slots. In chaining, α represents the **average length of a chain**.

3.2 Complexity Analysis under SUHA

Under the **Simple Uniform Hashing Assumption (SUHA)** (any key is equally likely to hash to any slot, independently of other keys):

- 1. Unsuccessful Search:** Expected time is $\Theta(1 + \alpha)$.
 - Computing $h(k)$ takes $O(1)$.
 - The entire chain at slot $h(k)$ must be scanned to its end. Expected chain length is α .
- 2. Successful Search:** Expected time is $\Theta\left(1 + \frac{\alpha}{2}\right)$.
 - The key searched for is equally likely to be anywhere in its chain. On average, half the chain is traversed.
- 3. Worst Case:** $\Theta(n)$.
 - Occurs when all n keys hash to the exact same slot, reducing the table to a single linked list.
- 4. Insertion:** $O(1)$ if inserted at the head of the list; $O(1 + \alpha)$ if verifying uniqueness.
- 5. Deletion:** $O(1)$ if given the node pointer in a doubly linked list; $O(1 + \alpha)$ in a singly linked list.

3.3 Python Implementation

```
class SeparateChainingTable:
    def __init__(self, capacity: int = 7):
        self.capacity = capacity
        self.table = [[] for _ in range(capacity)]
```

```

def _hash(self, key: int) -> int:
    return key % self.capacity

def insert(self, key: int, value: str) -> None:
    slot = self._hash(key)
    for i, (k, v) in enumerate(self.table[slot]):
        if k == key:
            self.table[slot][i] = (key, value) # Update
            return
    self.table[slot].append((key, value))      # O(1) append

def search(self, key: int):
    slot = self._hash(key)
    for k, v in self.table[slot]:
        if k == key:
            return v
    return None

def delete(self, key: int) -> bool:
    slot = self._hash(key)
    for i, (k, v) in enumerate(self.table[slot]):
        if k == key:
            self.table[slot].pop(i)
            return True
    return False

```

4. Strategy 2: Open Addressing (Closed Hashing)

In **Open Addressing**, all elements reside directly within the array $T[0 \dots m-1]$. When a collision occurs, alternate slots are probed until an empty slot is located.

General probe equation:

$$h(k, i) = (h'(k) + f(i)) \bmod m \quad \text{for probe number } i \in \{0, 1, \dots, m-1\}$$

Because every key occupies a table slot, the load factor cannot exceed 1:

$$\alpha = \frac{n}{m} \leq 1$$

4.1 Theoretical Bounds under Uniform Hashing

Assuming every probe sequence is equally likely among all $m!$ permutations:

Operation	Expected Probes (Upper Bound)	Value at $\alpha = 0.5$	Value at $\alpha = 0.9$
Unsuccessful Search / Insert	$\frac{1}{1 - \alpha}$	2.0 probes	10.0 probes
Successful Search	$\frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right)$	≈ 1.39 probes	≈ 2.56 probes

5. Probing Techniques in Open Addressing

5.1 Linear Probing

Formula

Offset function is linear: $f(i) = i$:

$$h(k, i) = (h'(k) + i) \bmod m$$

Distinct Probe Sequences

There are only **m distinct probe sequences**. The entire sequence is determined solely by the initial probe $h'(k)$.

Primary Clustering

Linear probing suffers from **Primary Clustering**:

- Occupied slots form contiguous blocks.
- Any key hashing to any slot inside a cluster—or to the empty slot right after it—will extend that cluster.
- Probability that a cluster of length L grows during the next insertion is:

$$P(\text{cluster grows}) = \frac{L + 1}{m}$$

Long clusters attract more incoming keys and snowball into even larger clusters.

Expected Probes (Knuth's Formulas)

- **Unsuccessful Search:** $\approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2}\right)$
 - **Successful Search:** $\approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha}\right)$
-

5.2 Quadratic Probing

Formula

Offset function is quadratic: $f(i) = c_1 i + c_2 i^2$ (commonly $c_1 = 0, c_2 = 1$):

$$h(k, i) = (h'(k) + i^2) \bmod m$$

Elimination of Primary Clustering & Secondary Clustering

- **Primary clustering is eliminated** because step sizes increase quadratically ($+1, +4, +9, \dots$), breaking contiguous runs.
- **Secondary Clustering:** If two keys have the same initial hash ($h'(k_1) = h'(k_2)$), they generate the **exact same probe sequence**.
- Number of distinct probe sequences: still only **m** .

Coverage Theorem

If table size m is a prime number and $h(k, i) = (h'(k) + i^2) \bmod m$, the first $\lceil m/2 \rceil$ probe locations are distinct. Consequently, an empty slot is **guaranteed to be found as long as $\alpha < 0.5$** .

5.3 Double Hashing

Formula

Offset function uses a second hash function $h_2(k)$ multiplied by probe index i :

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Distinct Probe Sequences

Because the step size depends on key k , the probe sequence depends on both $h_1(k)$ and $h_2(k)$. This produces $\Theta(m^2)$ **distinct probe sequences**, offering the closest practical approximation to uniform hashing.

Strict Requirements on $h_2(k)$

1. **$h_2(k) \neq 0$** : If $h_2(k) = 0$, the step size is zero, trapping search in an infinite loop at $h_1(k)$.
2. **$\gcd(h_2(k), m) = 1$** : $h_2(k)$ must be relatively prime to m so that the probe sequence visits all m slots before repeating.

Standard Design:

- Choose m as a prime number.
 - $h_1(k) = k \bmod m$
 - $h_2(k) = 1 + (k \bmod (m - 1))$ (Guarantees $h_2(k) \in [1, m - 1]$, always coprime to prime m).
-

6. Deletion in Open Addressing: The Tombstone Mechanism

6.1 Why Naive Deletion Fails

In open addressing, an unsuccessful search terminates immediately when it hits an empty slot (None). If key k_2 is deleted by setting its slot to None, any subsequent search for key k_3 (which probed past k_2 during insertion) will hit None and falsely report that k_3 does not exist.

Initial (keys k_1, k_2, k_3 all hash to 3):

Slot:	0	1	2	3	4	5	6
Table:	[]	[]	[]	[k_1]	[k_2]	[k_3]	[]

Naive deletion of k_2 sets slot 4 to None:

Table: [] [] [] [k1] [None] [k3] []



Search for k3 starts at 3, checks 4, sees None, and aborts! (k3 is lost)

6.2 Solution: Lazy Deletion (Tombstones)

Mark deleted slots with a special sentinel value: DELETED (tombstone).

- **Search:** Treat DELETED as occupied; continue probing. Stop only on target key or None.
- **Insert:** Can reuse the first DELETED slot encountered once it confirms key is not in the table.
- **Delete:** Replace the target slot with DELETED.

7. Step-by-Step Insertion Traces (GATE High Yield)

Consider table size $m = 7$. Insert keys: $[10, 22, 31, 4, 15, 28, 17]$ using base hash $h(k) = k \bmod 7$.

- $h(10) = 3$
- $h(22) = 1$
- $h(31) = 3$ (collides with 10)
- $h(4) = 4$
- $h(15) = 1$ (collides with 22)
- $h(28) = 0$
- $h(17) = 3$ (collides with 10, 31)

7.1 Linear Probing Trace

Formula: $h(k, i) = (h(k) + i) \bmod 7$

Key	Home $h(k)$	Probe Path Checked	Probes	Placed Slot	Table State $[0, 1, 2, 3, 4, 5, 6]$
10	3	[3]	1	3	[_, _, _, 10, _, _, _]
22	1	[1]	1	1	[_, 22, _, 10, _, _, _]
31	3	[3:occ] $\rightarrow$ [4:free]	2	4	[_, 22, _, 10, 31, _, _]
4	4	[4:occ] $\rightarrow$ [5:free]	2	5	[_, 22, _, 10, 31, 4, _]
15	1	[1:occ] $\rightarrow$ [2:free]	2	2	[_, 22, 15, 10, 31, 4, _]
28	0	[0]	1	0	[28, 22, 15, 10, 31, 4, _]
17	3	[3:occ] $\rightarrow$ [4:occ] $\rightarrow$ [5:occ] $\rightarrow$ [6:free]	4	6	[28, 22, 15, 10, 31, 4, 17]

Total probes: $1 + 1 + 2 + 2 + 2 + 1 + 4 = \mathbf{13}$ probes.

7.2 Quadratic Probing Trace

Formula: $h(k, i) = (h(k) + i^2) \bmod 7$

Key	Home $h(k)$	Probe Calculations	Probes	Placed Slot	Table State $[0, 1, 2, 3, 4, 5, 6]$
10	3	$i=0: 3$	1	3	$[_ , _ , _ , 10 , _ , _]$
22	1	$i=0: 1$	1	1	$[_ , 22 , _ , 10 , _ , _]$
31	3	$i=0: 3$ (occ) $\rightarrow i=1: (3+1)\%7 = \mathbf{4}$	2	4	$[_ , 22 , _ , 10 , 31 , _]$
4	4	$i=0: 4$ (occ) $\rightarrow i=1: (4+1)\%7 = \mathbf{5}$	2	5	$[_ , 22 , _ , 10 , 31 , 4 , _]$
15	1	$i=0: 1$ (occ) $\rightarrow i=1: (1+1)\%7 = \mathbf{2}$	2	2	$[_ , 22 , 15 , 10 , 31 , 4 , _]$
28	0	$i=0: 0$	1	0	$[28 , 22 , 15 , 10 , 31 , 4 , _]$
17	3	$i=0: 3$ (occ) $\rightarrow i=1: 4$ (occ) $\rightarrow i=2: (3+4)\%7 = 0$ (occ) $\rightarrow i=3: (3+9)\%7 = \mathbf{5}$ (occ)	—	—	Note: for $\alpha = 1$, quadratic probing may cycle without visiting all slots.

7.3 Double Hashing Trace

Formulas:

- $h_1(k) = k \bmod 7$
- $h_2(k) = 1 + (k \bmod 5)$

Step sizes:

- $h_2(10) = 1 + 0 = \mathbf{1}$
- $h_2(22) = 1 + 2 = \mathbf{3}$
- $h_2(31) = 1 + 1 = \mathbf{2}$
- $h_2(4) = 1 + 4 = \mathbf{5}$
- $h_2(15) = 1 + 0 = \mathbf{1}$
- $h_2(28) = 1 + 3 = \mathbf{4}$
- $h_2(17) = 1 + 2 = \mathbf{3}$

Key	$h_1(k)$	Step $h_2(k)$	Probe Calculations	Probes	Placed Slot	Table State $[0, 1, 2, 3, 4, 5, 6]$
10	3	1	$i=0: \mathbf{3}$	1	3	$[_ , _ , _ , 10 , _ , _]$
22	1	3	$i=0: \mathbf{1}$	1	1	$[_ , 22 , _ , 10 , _ , _]$

31	3	2	$i=0: 3\$ (occ) \$\to i=1: (3+2)\%7 = \mathbf{5}\$$	2	5	[_, 22, _, 10, _, 31, _]
4	4	5	$i=0: \mathbf{4}\$$	1	4	[_, 22, _, 10, 4, 31, _]
15	1	1	$i=0: 1\$ (occ) \$\to i=1: (1+1)\%7 = \mathbf{2}\$$	2	2	[_, 22, 15, 10, 4, 31, _]
28	0	4	$i=0: \mathbf{0}\$$	1	0	[28, 22, 15, 10, 4, 31, _]
17	3	3	$i=0: 3\$ (occ) \$\to i=1: (3+3)\%7 = \mathbf{6}\$$	2	6	[28, 22, 15, 10, 4, 31, 17]

Total probes: $1 + 1 + 2 + 1 + 2 + 1 + 2 = \mathbf{10}\$$ probes.

[!TIP] Notice key 17: under Linear Probing it took **4 probes** due to primary clustering. Under Double Hashing it took only **2 probes**.

8. Master Comparative Summary Table

Feature	Separate Chaining	Linear Probing	Quadratic Probing	Double Hashing
Storage Model	Auxiliary linked lists	Inside array	Inside array	Inside array
Load Factor Bound	Unbounded ($\alpha > 1\$$ allowed)	Strictly $\alpha \leq 1\$$	Strictly $\alpha \leq 1\$$ ($\alpha < 0.5\$$ for coverage)	Strictly $\alpha \leq 1\$$
Number of Probe Sequences	$m\$$ buckets	$m\$$ sequences	$m\$$ sequences	$\Theta(m^2)\$$ sequences
Primary Clustering	None	Severe	None	None
Secondary Clustering	None	Inherent	Yes	None
Expected Unsuccessful Search	$\Theta(1 + \alpha)\$$	$\approx \frac{1}{2}(1 + \frac{1}{1-\alpha^2})\$$	$\approx \frac{1}{1-\alpha}\$$	$\frac{1}{1-\alpha}\$$
Expected Successful Search	$\Theta(1 + \frac{\alpha}{2})\$$	$\approx \frac{1}{2}(1 + \frac{1}{1-\alpha})\$$	$\approx \frac{1}{\alpha \ln \frac{1}{1-\alpha}}\$$	$\frac{1}{\alpha \ln \frac{1}{1-\alpha}}\$$
Worst-Case Search	$\Theta(n)\$$	$\Theta(m)\$$	$\Theta(m)\$$	$\Theta(m)\$$
Deletion Mechanism	Standard pointer unlinking	Tombstones / Lazy deletion	Tombstones / Lazy deletion	Tombstones / Lazy deletion

9. Common GATE Exam Traps & Practice Questions

Exam Trap 1: Zero Step Size in Double Hashing

[!CAUTION] In double hashing $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, why can $h_2(k)$ never evaluate to 0? **Answer:** If $h_2(k) = 0$, $i \cdot h_2(k) = 0$ for all i . The formula reduces to $h(k, i) = h_1(k)$ for all probes, entering an **infinite loop** checking the same slot repeatedly.

Exam Trap 2: Distinct Probe Sequences Count

- Linear Probing: m distinct sequences.
 - Quadratic Probing: m distinct sequences.
 - Double Hashing: $\Theta(m^2)$ distinct sequences.
-

Practice Question 1 (GATE CS Numerical)

Question: Consider a hash table of size $m = 11$ with hash function $h(k) = k \bmod 11$. The keys 43, 23, 1, 62, 34, 28, 14 are inserted using **Linear Probing**. Find:

1. The slot occupied by key 14.
2. The total number of collisions during insertion.

Solution:

1. $h(43) = 43 \bmod 11 = 10 \rightarrow$ Slot 10 (0 collisions)
 2. $h(23) = 23 \bmod 11 = 1 \rightarrow$ Slot 1 (0 collisions)
 3. $h(1) = 1 \bmod 11 = 1 \rightarrow$ Slot 1 occupied $\rightarrow$ Slot 2 (1 collision)
 4. $h(62) = 62 \bmod 11 = 7 \rightarrow$ Slot 7 (0 collisions)
 5. $h(34) = 34 \bmod 11 = 1 \rightarrow$ Slot 1, 2 occupied $\rightarrow$ Slot 3 (2 collisions)
 6. $h(28) = 28 \bmod 11 = 6 \rightarrow$ Slot 6 (0 collisions)
 7. $h(14) = 14 \bmod 11 = 3 \rightarrow$ Slot 3 occupied $\rightarrow$ Slot 4 (1 collision)
- Key 14 is stored at **Slot 4**.
 - Total collisions = $0 + 0 + 1 + 0 + 2 + 0 + 1 = \mathbf{4}$.
-

Practice Question 2 (GATE CS Conceptual)

Question: An empty hash table of size 7 uses open addressing with hash function $h(k) = k \bmod 7$. If the load factor $\alpha = \frac{5}{7}$, what is the maximum number of probe sequences possible if: (a) Linear Probing is used? (b) Double Hashing is used?

Answer:

- (a) Linear Probing: Exactly $m = \mathbf{7}$ probe sequences.
 - (b) Double Hashing: Up to $m(m-1) = 7 \times 6 = \mathbf{42}$ (or $\Theta(m^2)$) distinct probe sequences.
-

Summary

- **Collision Inevitability:** By the Pigeonhole Principle and Birthday Paradox, collisions are guaranteed even at small load factors.
 - **Terminology:**
 - Separate Chaining $\equiv$ Open Hashing $\equiv$ Closed Addressing (α unbounded).
 - Open Addressing $\equiv$ Closed Hashing ($\alpha \leq 1$).
 - **Separate Chaining:** Expected unsuccessful search is $\Theta(1 + \alpha)$; successful search is $\Theta(1 + \alpha/2)$.
 - **Linear Probing:** Suffers from **Primary Clustering**; produces m probe sequences.
 - **Quadratic Probing:** Eliminates primary clustering but exhibits **Secondary Clustering**; covers $\geq \lceil m/2 \rceil$ slots when m is prime.
 - **Double Hashing:** Eliminates both primary and secondary clustering by generating $\Theta(m^2)$ probe sequences. Requires $h_2(k) \neq 0$ and $\gcd(h_2(k), m) = 1$.
 - **Deletion in Open Addressing:** Requires **Tombstones** (DELETED) to avoid breaking the search probe chain.
-

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.), Chapter 11: Hash Tables. MIT Press.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.), Section 6.4: Hashing. Addison-Wesley.
- Skiena, S. S. (2020). *The Algorithm Design Manual* (3rd ed.), Section 3.7: Hashing. Springer.